Project - Low Level Design

on

IaC Provisioning for Agriculture System

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1)	Aditya Pandey	EN22CS301057
2)	Aaditya Verma	EN22CS301009
3)	Aditya Thakre	EN22CS301069
4)	Aakansha Rajani	EN22CS301012
5)	Abhijeet Patidar	EN22CS301029

Group Name: Group 11 D1

Project Number: DO-11

Industry Mentor Name:

University Mentor Name:

Academic Year: 2026

Table of Contents

S.no		Page No.
1.	Introduction	3
1.1	Scope of the document	3
1.2	Intended Audiences	4
1.3	System overview	4
2	System Design	5
2.1	Application Design	5
2.2	Process Flow	6
2.3	Information Flow	6
2.4	Component Design	7
2.5	Key Design Considerations	7
2.6	API Catalogue	8
3	References	8
4	GitHub	

1. Introduction

The project titled "Infrastructure as Code (IaC) Provisioning for Agriculture System (AgriTriage)" focuses on the design and deployment of an AI-powered platform that assists farmers by analyzing agricultural queries and generating intelligent recommendations. The system leverages modern Natural Language Processing techniques to interpret user inputs and provide structured, context-aware responses that can help in decision-making and problem resolution.

The primary objective of this project is to integrate application development with DevOps practices in order to achieve a streamlined and automated deployment process. The system is built using a FastAPI-based backend combined with an AI processing pipeline powered by LangChain and Groq Large Language Models. These technologies enable the system to perform advanced tasks such as classification of queries, extraction of relevant information, and generation of meaningful responses.

From an infrastructure perspective, the application is deployed on Amazon Web Services (AWS), where an EC2 instance is used as the hosting environment. The entire application is containerized using Docker, which ensures consistency across development and deployment environments. This approach simplifies dependency management, enhances portability, and allows the system to be deployed quickly and reliably.

1.1 Scope of the Document

This document presents the Low-Level Design (LLD) of the project titled "Infrastructure as Code (IaC) Provisioning for Agriculture System (AgriTriage)". The primary objective of this system is to design and deploy an AI-powered agriculture assistance platform while integrating modern DevOps practices using AWS cloud services and Docker-based containerization.

The document provides a detailed description of the internal architecture of the system, the interaction between different modules, and the deployment workflow followed for hosting the application on cloud infrastructure. It explains how the backend services, AI

processing modules, and deployment mechanisms are structured and how they collectively ensure efficient and reliable system behavior.

In addition, this document outlines the API structure, system components, and design decisions that were taken to achieve modularity, scalability, and maintainability. However, it does not include details related to internal AI model training or domain-specific agricultural logic, as the focus is primarily on system design and deployment.

1.2 Intended Audience

This document is intended for multiple stakeholders involved in the development and evaluation of the system. It serves as a reference for backend developers who need to understand the internal architecture and API flow of the application. DevOps engineers can use this document to understand the deployment process, containerization strategy, and cloud configuration. Cloud architects may refer to it for evaluating infrastructure decisions, while testers can use it to validate system behavior. Additionally, academic evaluators can assess the depth of implementation and adherence to software engineering principles through this document

1.3 System Overview

The AgriTriage System is an AI-driven agriculture support platform designed to assist farmers by analyzing their queries and providing intelligent recommendations. The system functions as a triage mechanism that interprets user input, determines the nature and urgency of the problem, and generates appropriate responses using Natural Language Processing techniques.

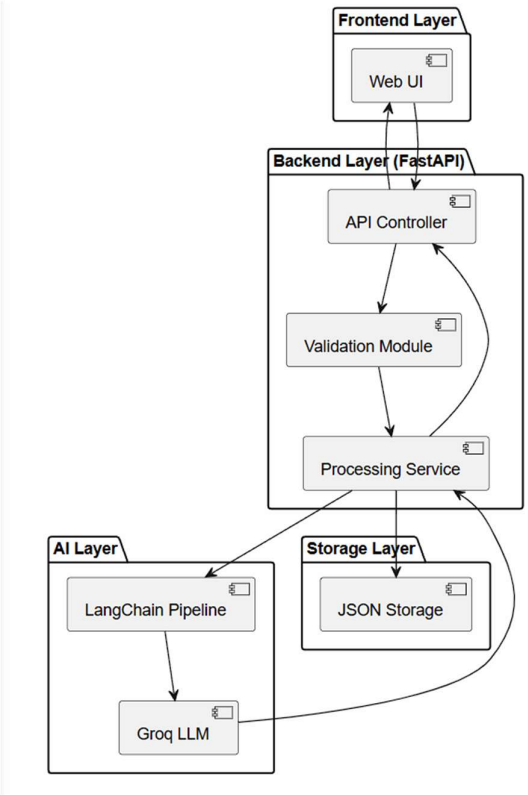
At its core, the system integrates a FastAPI-based backend with an AI processing pipeline powered by LangChain and Groq Large Language Models. The backend is responsible for handling incoming requests, validating inputs, and coordinating with the AI module to generate responses. The AI module processes the query in multiple stages, including classification, entity recognition, and response generation, ensuring that the output is both relevant and structured.

From a deployment perspective, the system is designed as a cloud-native application hosted on AWS. The application runs inside a Docker container, which ensures consistency across environments and simplifies deployment. The containerized application is deployed on an AWS EC2 instance, where it is exposed to users via a public IP address. This setup provides a balance between simplicity and scalability while maintaining control over infrastructure.

Overall, the system follows a modular architecture that separates concerns between the user interface, backend processing, AI logic, and deployment environment. This design allows the system to be easily extended or scaled in the future.

2. System Design

2.1 Application Design



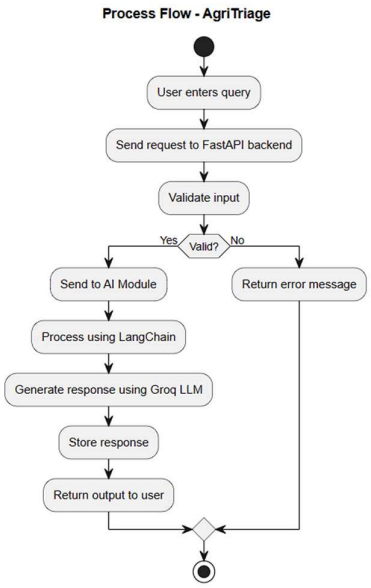
The application design of the AgriTriage system follows a layered architecture that separates user interaction, backend processing, artificial intelligence logic, and data storage into distinct modules. This design ensures modularity, maintainability, and ease of extension.

The frontend layer provides the user interface through which farmers can submit their agricultural queries. It communicates with the backend using HTTP-based requests. The backend layer, implemented using FastAPI, acts as the central processing unit of the application. It handles request routing, validates incoming data, and manages the overall workflow.

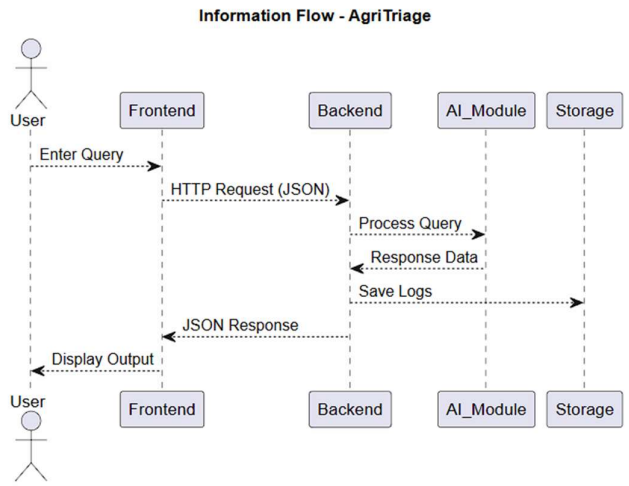
Once a request is validated, it is passed to the processing service, which interacts with the AI layer. The AI layer, powered by LangChain and Groq LLM, processes the input query through multiple stages, including classification, entity extraction, and response generation. The generated output is then sent back to the backend.

A storage layer is included to maintain logs of user queries and responses. This enables tracking of system activity and can be used for future improvements. The entire architecture is designed to ensure smooth communication between components while maintaining clear separation of responsibilities.

2.2 Process Flow



2.3 Information Flow



2.4 Components Design

The component design of the AgriTriage system focuses on clearly defining the responsibilities of each major element within the architecture. The system is deployed on an AWS EC2 instance, which serves as the hosting environment. Within this environment, the entire application is encapsulated inside a Docker container, ensuring consistency and portability.

The FastAPI backend acts as the central component that manages communication between the user interface and the AI module. It handles API requests, performs validation, and coordinates the processing workflow. The AI module, which includes LangChain and Groq LLM, is responsible for analyzing queries and generating intelligent responses.

A storage component is integrated to maintain logs of interactions, which helps in tracking system activity and improving performance over time. All these components operate within the Docker container, allowing the system to be easily deployed and managed.

This component-based design ensures that each part of the system has a well-defined role, making the architecture clean, scalable, and easy to maintain.

2.5 Key Design Considerations

The design of the AgriTriage system is guided by several important considerations. Scalability is achieved through the use of cloud infrastructure, allowing the system to be extended in the future by upgrading instance types or adding additional instances. Availability depends on the reliability of the AWS EC2 instance, which ensures continuous operation of the application.

Security is maintained through the use of AWS security groups and secure access mechanisms such as SSH key authentication. Sensitive information is not hardcoded and is managed securely within the environment.

Portability is ensured through Docker, which allows the application to run consistently across different platforms. The modular architecture improves maintainability by allowing individual components to be modified or replaced without affecting the entire system.

2.6 API Catalogue

The system exposes a set of REST APIs that enable communication between the frontend and backend. These APIs are designed to handle different aspects of the system functionality, including query submission, processing, and response generation.

The core APIs include endpoints for submitting agricultural queries, analyzing them using the AI module, and generating recommendations. In addition to these, system-level APIs are provided to monitor the health and status of the application. These APIs ensure that the system remains accessible, functional, and easy to integrate with other services if required.

3. References

1. Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education.
2. Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*. Pearson Education.
3. Sommerville, I. (2016). *Software Engineering*. Pearson.
4. IEEE Standards Association. *IEEE Recommended Practice for Software Design Descriptions (IEEE 1016)*.
5. Food and Agriculture Organization (FAO).
   Official agricultural knowledge resources and datasets.
   https://www.fao.org